身知
BodyLens

BODY LEDGER

把抽烟、喝酒、熬夜、久坐这些每天都在发生的小事，
换算成「今天从你身上拿走了多少时间」，记在一本账上。

文档类型	技术文档 · 产品与工程说明
在线演示	https://body-ledger.com
代码仓库	github.com/asval101/bodylens（公开）
技术形态	原生 HTML / CSS / JavaScript，零运行时依赖，无构建、无框架、无 CDN
规模	86 次提交 · 30 条行为 · 10 个器官 · 12 份来源 · 85 项自动化测试
日期	2026 年 10 月 28 日

CONTENTS

目录

00 摘要 这个产品解决什么问题，以及本文档想证明什么 3

01 问题与定位 从一句听了无数遍却没有用的话开始 4

1.1 起点 · 1.2 同类产品的共同错误 · 1.3 换成结账 · 1.4 用户与场景 · 1.5 砍掉的四件事 4

02 技术栈：为什么是原生 一个不流行但被评分表逼出来的选择 6

2.1 不推荐的选择 · 2.2 四条硬指标 · 2.3 零依赖长什么样 · 2.4 代价与补救 · 2.5 唯一例外 6

03 架构与数据流 一份权重表、一处转换、一条不许越界的表现层 9

3.1 分层 · 3.2 单一真相源 · 3.3 唯一一处转换 · 3.4 推演不是 AI · 3.5 表现层边界 · 3.6 为什么合成一页 9

04 数字口径：三条铁律 写在代码里，不是写在文档里 12

4.1 有数字必有 caveat · 4.2 推算要写路径 · 4.3 只在尺度内成立 · 4.4 三个不做 12

05 意图控制：AI 协同怎么做的 三个案例、一份被否清单、一套验收方法 14

5.1 角色分工 · 5.2 证据怎么留 · 5.3–5.5 三个案例 · 5.6 被否掉的提议 · 5.7 我怎么验收 14

06 演进时间线 十一天、98 次提交，每一次都能独立说明 19

07 质量保障 不看「它说完成了」，看像素和测试 20

7.1 单元测试 · 7.2 无头浏览器探针 · 7.3 逐帧验证 · 7.4 像素级验收 20

08 部署与复现 十分钟从零到页面能跑 23

09 已知局限 我自己知道有问题、且没解决的地方 25

A 附录：验收标准对照 PRD 的 28 条验收项，逐条给当前状态 27

怎么读这份文档

如果你只有五分钟：读第 00 节和第 05 节。前者说这个产品是什么，后者说 AI 在其中扮演什么角色——那是本次比赛评分里占 45% 的部分。

如果你想验证我说的话：所有数字都能在仓库里查到。文中的提交号可以直接 `git show`，测试命令是 `node --test`，线上地址可以打开。

00 / SUMMARY

摘要

身知 BodyLens 是一本记录日常行为的账。 你抽了几支烟、喝了几杯酒、少睡了几小时，它不告诉你「哪里坏了」，而是算给你看：今天从你身上拿走了多少时间。这个数字会累积、可比较、能换算成你熟悉的事——二十支烟大约一百零一天。

0.1 做了什么

一个单页 Web 应用。左侧是今天记的账，右侧是一具可点的人体。点一笔行为即入账，数字立刻变化，涉及的器官同时亮起；点某个器官，看到它今天和这些行为是什么关系、依据哪份研究、哪些地方还没有数据。往下滚是「如果一直这样」的推演：同一份权重换一个时间尺度，看一周、一个月、一年。

所有内容离线可用。全站零运行时依赖——没有 React、没有构建步骤、没有 CDN、没有在线字体。在线大模型只负责把已经算好的数字换一种说法，接不上就退回本地规则，功能不缺一项。

0.2 最难的地方

不是画人体，也不是做动画。难的是**让每一个数字都站得住**。

这个产品要说「一支烟等于 20 分钟」，而这 20 分钟是我从一个折算口径里读出来的，不是任何一份研究里写着的一句话。一旦我把推算出来的数字和原文给的数字混在一起显示，产品就从「帮人理解」变成了「伪造精度」。所以数值层被定成了三条硬规则，并且写成了会红的测试：有数字必须有说明、推算必须标出路径、只在推演尺度内成立的数才允许进推演。

第二条难点是**克制**。过程中砍掉的东西比留下的多：身体年龄做过一版又砍了，健康总分砍了，损伤百分比砍了。每一次砍都在削弱产品的「直观程度」，但每一次都在补「可信程度」。这个取舍的详细记录在 `docs/DESIGN-INTENT.md`。

0.3 怎么证明

代码可查	86 次提交，从 2026-10-18 到 2026-10-26，没有一次「initial commit 含全部功能」。
过程可查	三个核心迭代的完整 Prompt 链带公开分享页、图文归档与全文抽取，时间戳可独立核验。
行为可查	85 项 `node:test` 用例，覆盖折算、周额度、跨午夜、备份校验、离线降级。
结果可查	线上 `body-ledger.com`，断网后核心功能仍可用（AC-24）。

一句话

这个项目的赌注是：**可信比好看来得值钱。**所以它的技术选择、视觉选择和文案选择，指向的其实是同一件事——不让用户产生「它好像很懂我的身体」的错觉。

01 / PROBLEM

问题与定位

健康类产品几乎都做了同一件事：**告诉你哪里坏了。** 打分、标红、出报告。它们描述的是一个状态，而这个状态我既无法验证，也无法据此决定今天做什么。

1.1 起点：一句听了无数遍、却没有任何用的话

不是从「健康管理市场很大」开始的。是从这句话开始的：

> 「每天一包烟，少活十年。」

我知道它大概是真的。但它不构成任何行动。

原因不在于信息不够，而在于**尺度和我的经验对不上。**「十年」在时间上是抽象的，在概率上是群体的，在责任上是遥远的。它既不能让我今天少抽一根，也不能让我在抽完之后有半点具体的感受。

1.2 同类产品的共同错误

它们都选了同一个方向：告诉你哪里坏了。

三种常见做法的失效原因

做法	典型表达	为什么没用
给一个分数	健康指数 72	不知道 72 和 68 差在哪，也不知道它凭什么这么算。
画一个红点	你的肝：⚠️	这是恐吓。恐吓的有效期只有三天。
出体检式报告	肺功能下降 12%	我没有基线，也无从验证。而且这个数字通常无法验证。

三者的共同点是：**它们都在描述一个状态，而不是在结算一笔交易。**状态不可累积，也没有进度感——看完就结束了，第二天什么都不剩。

1.3 换机制：用时间结账

第一版做的是最直觉的那件事：选行为 → 点亮器官 → 讲器官受什么影响。做完看了一遍，发现一个问题：**它是个解释器，不是个产品。**用户点完「抽烟」，看到肺、心、脑、口腔一起亮，读了三段有出处的话，然后呢？他会觉得「哦，有道理」，然后关掉页面。

于是换了机制：

```
一条行为          →  折算成一个分钟数（负数 = 损耗）
今天所有条目      →  相加            →  今天从你这里拿走的时间
同一份权重        →  换一个时间尺度  →  如果一直这样：1 周 / 1 个月 / 1 年
```

为什么是时间，四个理由：

- **所有人都有刻度。**你有 24 小时，你花掉它们，也浪费它们。不需要学习就能理解。
- **它能同时对上「一次」和「一辈子」。**一支烟 20 分钟，一包 400 分钟，一年约 101 天。同一把尺子，从行为量到人生。
- **它可比较。**一杯酒 15 分钟、一小时屏幕 21.8 分钟、一小时睡眠不足 11 分钟——它们能排在同一栏里。「肝受到的损伤」和「肺受到的损伤」不能。
- **数字是负的，但产品不悲惨。**账本是正是负由用户自己填，系统只结算，不评价。

1.4 用户与场景

目标用户是**知道自己该改、但一直没动**的那批人：二十到三十五岁，抽一点烟、熬一点夜、坐得很久，看过很多科普，都不为所动。他们不需要更多知识，需要一个能看见累积的计数器。

1.5 砍掉的四件事

取舍记录比功能清单更能说明意图。以下四个功能都做过或设计过，最后砍了：

主动砍掉的功能及原因

砍掉的	原因
身体年龄	做过一版，砍了。它不是可验证的量；且一旦给出，用户会追问「那我还能活多久」——那是另一个问题，而且不可回答。
健康总分	把抽烟的害和运动的益压成一个数，等于替用户做了他没有授权的价值判断。
损伤百分比、病理解剖图	输入只有「每天几支」，输出一张黑肺，是伪造精度。
步数式打卡	会把注意力从「账」转移到「连胜」，产品变成游戏。

> **注意**
>
> 砍掉「身体年龄」的代价是：产品失去了唯一一个普通人能瞬间理解的总量指标。这是有意识的让步——「负荷 61/100」缺乏体感，但至少不会撒谎；身体年龄有体感，但无法验证。最终选了前者，因为它可比较、有区间、不需要医学解释。

02 / STACK

技术栈：为什么是原生

官方推荐的技术栈是 React / Vue / Node。**我没有用。** 这不是为了显得特别，而是因为我算过账：换框架会让两条硬指标同时作废，而它们正好是评分表上「过程可复现」那 25 分的抓手。

2.1 一个不推荐的选择

先说清楚代价，再说理由。用原生意味着：没有组件化、没有类型检查、没有热更新、没有生态。所有状态变更靠手写 DOM 操作，所有复用靠函数和模板字符串。

在正常的产品环境里，这是一笔坏交易。但这个项目的评分表里，「过程可复现与演进链路」占 25%、「工程质量与架构」占 15%，而这两项考的都不是技术时髦度，是**别人能不能在你的仓库里把东西跑起来、并且看懂你为什么这么写。**

2.2 四条硬指标怎么定死了技术栈

PRD 中的四条约束，与框架方案的冲突

编号	要求	为什么框架方案会失败
AC-24	断网后核心功能可用	框架本身要从 CDN 或 node_modules 加载；一旦要求「零外部请求」，就得把框架内联进来，等于给自己造一个 300KB 的行李箱。
AC-28	约 10 分钟启动，无付费密钥	Next.js 需要 `npm install`（数百个包）+ 构建 + 环境变量。更换机器时的不确定性，会直接吃掉复现演示的时间。
零支出	不产生任何持续费用	框架方案的默认部署目标是托管平台，免费额度与域名绑定有限制。
可追溯	评委能在本机跑起来	46 个文件重写是纯损耗，且会让已有的提交历史失去意义。

结论很干脆：**换框架会让这四条同时作废，而它们正是我要拿分的地方。**技术栈在这里不是审美问题，是策略问题。

2.3 零依赖长什么样

具体做法只有三条规矩，全站遵守：

- **模块方式：全局 IIFE + 命名空间。**每个文件是一个立即执行函数，把公开接口挂到 `window.BodyLens` 上。没有 `import`，没有打包器，`index.html` 里按依赖顺序写 `<script>` 即可。
- **模板方式：字符串 + 事件委托。**所有列表用模板字符串生成，事件统一挂在容器上。这比手写 `createElement` 短，比引入模板引擎轻。

— **资源方式：全部本地。**字体用系统字体栈，人体是内联 SVG，纹理是本地图片。全站没有一次跨域请求（AI 接口除外，且是可选的同源 `/api/interpret`）。

```
(function (global) {
  'use strict';
  const W = global.BodyLensWeights;      // 依赖通过命名空间取，不通过 import

  function summarize(entries) { /* 纯函数：账本 → 合计 */ }

  global.BodyLensLedger = { summarize, /* ... */ };
})(typeof window === 'undefined' ? globalThis : window);
```

2.4 代价与补救

没有框架兜底，就得自己补三样东西。这三样恰好也是这个项目里质量最扎实的部分：

缺少框架能力之后的替代方案

缺什么	怎么补	效果
组件化	把「会变的逻辑」全部下沉成纯函数模块：`ledger.js`、`forecast.js`、`domain.js`、`interpret.js`。DOM 只在 `app.js` 和 `today.js` 里出现。	计算层不依赖 DOM，可以直接被 `node:test` 调用，85 项用例因此完全不需要浏览器。
类型检查	数据层自带 `validate()`：逐条检查来源是否登记、证据等级是否合法、有数字是否带 caveat，测试要求它返回空数组。	数据错误挡在提交前，而不是挡在用户面前。
状态管理	只有一份 `state` 对象，序列化成 `{schemaVersion, dataVersion, data}` 存进 localStorage，备份和恢复都走这一个入口。	没有「两套状态不同步」这类问题——这类问题在原生产品里最常见。

为什么把「不许新增超过 150 行」写进规范

`AGENTS.md` 里有一条：`app.js` 不得再新增超过 150 行功能代码，新功能一律建独立文件。理由不是洁癖——是因为单文件继续膨胀，会让 AI 每次改动的上下文成本失控，也让 diff 无法评审。**这是给 AI 协同留的接口，不是给人留的纪律。**

2.5 唯一例外：AI 措辞增强

全站只有一处联网：解读卡片的措辞。服务端一个约一百行的 Node 脚本（`server/proxy.cjs`）持有密钥并调用大模型，前端只请求同源的 `/api/interpret`。

三条边界不能破：

1. **Key 不进浏览器。**前端不知道密钥存在，也不直连任何第三方 API。

2. **AI 不算数。**它只负责把已经算好的数字换一种说法。所有数值由 `ledger.js` 和 `forecast.js` 在本地算出，AI 的输出里不出现新的数字。
3. **接不上就退回本地。**没有网络、没有密钥、限流、超时，一律返回本地规则文案，标签从「AI 解读」变回「本地解读」。**降级路径是默认路径，在线只是增强。**

这不是给产品加 AI 噱头，而是刻意把 AI 放在一个**做错了也不会造成事实错误**的位置上。设计取舍见 `docs/DECISIONS.md` 2026-10-20 第二条。

03 / ARCHITECTURE

架构与数据流

整个应用只有一条主干：**权重表 → 账本 → 界面。** 所有复杂度都被压到这条线以外——因为一旦有第二处地方也能算出「今天多少分钟」，两边迟早会不一样。

3.1 四层，越往下越纯

分层与职责

层	文件	职责与约束
数据层	`data/catalog.js` `data/weights.js`	只读。行为、器官、影响、来源、折算权重。唯一真相源，只有人能改。`weights.js` 自带 `validate()` 自检。

计算层	`ledger.js`
	`forecast.js`
	`interpret.js`
	`domain.js`
		纯函数，不碰 DOM、不碰网络、不碰 localStorage。因此可以在 Node 里被测试直接调用。

编排层	`app.js` `today.js`	唯一读写 `state` 的地方，唯一操作 DOM 的地方。负责页面切换、事件委托、持久化。
表现层	`vfx.js` `organ-focus.js`	只读 DOM 和 `body` 上的开关类。**不允许读写业务状态**，不允许碰 `state` / `localStorage` / `catalog`。

这条纪律不是形式主义。它带来一个具体的好处：**改视觉永远改不坏数字。**整个项目里视觉返工了十几轮（深色翻浅色、两页合并、人体重画、弹窗重做），计算层的测试一次都没红过。

3.2 单一真相源：`weights.js`

所有「一次行为等于多少分钟」只在这一个文件里定义，并且每条必须带出处与限制：

```
{
  id: 'sleep',
  minutes: 11,                    // 每少睡 1 小时
  range: [5, 17],                 // 区间必须一起给
  source: 'tsinghua-sleep',
  grade: 'B',
  hint: '推算值 · 长期少睡才成立',
  caveat: '这是推算值，不是原文直接给的数字：按 20–24 岁人群寿命损失约 0.94 年、
           持续 60 年折算，约合每天 22 分钟，即每少睡 1 小时约 11 分钟，区间 5–17 分钟。
           三个前提——一是只在长期维持时成立，偶发熬夜不适用；二是睡眠与死亡率呈 U 型
           关系，每天超过 9 小时风险同样上升；三是原研究为观察性，未证明因果。'
}
```

`validate()` 会逐条检查：来源是否在 `sources` 里登记、证据等级是否合法、有数字是否带 `caveat`。测试 `tests/ledger.test.cjs` 里有两条断言要求它返回空数组——**谁想加一个没有说明的数字，他会先看到测试红。**

3.3 全站唯一一处转换

今天的账本是「时间」，行为资料库要的是「情境」。这两个结构之间的转换只允许发生在一个地方：

```
// today.js:75
function scenario() {           // 账本 → 资料库情境
  // 按 itemId 累加，不取单笔最大值
}

// app.js:34
const appliedChoices = () => {  // 资料库唯一的读取入口
  return todayScenario() || [...];
};
```

上一版有两套数据：今天页读真实账本，身体探索页读一份写死的情境（默认「抽烟 1 支 / 1 个月」）。结果是——你在今天页记了十支烟，切过去它还是「一支一个月」。**用户说的「割裂」，看起来是长得不像，实际是两套数据。**详见 5.5 节。

3.4 推演不是 AI

「如果一直这样」那张卡上的数字，全部由 `forecast.js` 在本地算出来：把权重表的分钟数 × 数量 × 天数逐日累加。一次一年期的推演耗时约 2.5 毫秒。**AI 只负责换一种说法，一个数字都不碰。**

更关键的是准入白名单：

```
const ELIGIBLE = ['smoke', 'alcohol', 'sit'];
```

三十条行为里只有三条能进推演。睡眠那个 11 分钟是按 60 年摊出来的，而推演最多看一年——数字一样，含义差两个数量级。含糖饮料和快走则连分钟数都没有。**推演里出现一个编出来的数字，比不出现更糟。**

3.5 表现层的边界，用规则而不是自觉来守

`vfx.js` 提供分层景深、指针柔光、器官聚焦和点击脉冲。它的三条约束写在 `AGENTS.md` 里，改动前必须读：

- 不得读写业务状态，只允许读 DOM 与 `body` 上的 `no-motion` / `no-depth` 类；
- `app.js` 里不得新增视觉动效，新增一律进 `vfx.js`；
- 新增动效必须落在既有两个开关下——`body.no-motion` 关全部动画，`prefers-reduced-motion` 等同。

实际发生过一次越界：给人体某个 `<g>` 加了 SVG 的 `transform` 属性，结果 CSS hover 的 `scale()` 把它覆盖了，鼠标一上去头就跳。**坐标必须算进 path，不能靠属性叠加。**这条后来写进了项目笔记，因为它违反的不是某一次操作，而是「坐标算进 path」这条规矩。

3.6 为什么最终只剩一页

> 能不能把今天和身体探索两个部分合并？不要今天的那个身体了 因为我看「今天」这个页面比较空，而「身体探索」又有过多的信息了
>
> 用户，2026-10-21 17:41

合并之后：左侧是账，右侧是身体（sticky，内部滚动），1140px 以下切成单栏。导航只剩三项：今天 / 行为资料库 / 今日计划。

这次合并顺手删掉了一批东西：设定面板、器官名字标签、系统筛选、躯干切换、网格线、光斑、沉浸视图。删的理由都一样——**它们在替用户决定看什么。**全站样式表现在 1300 行左右，反而更清楚了。

04 / NUMBERS

数字口径：三条铁律

这个产品的分数里有一半靠「不把推测表现成已测量的事实」。 所以数值层定了三条规则，**写在代码里，不是写在文档里**——文档会过期，测试不会。

4.1 铁律一：有数字，就必须有 caveat

`data/weights.js` 里的 `validate()` 逐条检查数据。它不是运行时抛错，而是由测试守门：

```
// data/weights.js:231
if (typeof item.minutes === 'number' && !item.caveat) {
  problems.push(id + '：有数字但缺限制说明');
}

// tests/ledger.test.cjs
assert.deepStrictEqual(W.validate(), []);
```

也就是说，**错误挡在提交前，而不是挡在用户面前。**这是刻意的选择：一个没有说明的数字，在界面上看起来和一个有说明的数字一模一样，但前者是伪造精度。

4.2 铁律二：推算出来的数，必须写明推算路径

睡眠那一项最能说明问题。原文研究给的是「20–24 岁人群寿命损失约 0.94 年」，**没有**给「每少睡一小时等于多少分钟」。这个 11 分钟是折算出来的：

睡眠折算的四步，每一步都能复算

步	内容	结果
1	原文给出的寿命损失（YLL，20–24 岁，清华大学 & 哈佛，**Journal of Global Health**，2010–18 三轮 CFPS，每轮样本 2.8–3.1 万人）	男 0.94 年 / 女 0.73 年
2	摊到 60 年	约每天 22 分钟
3	除以「短睡眠」的 2 小时缺口	每少睡 1 小时约 11 分钟
4	按原文 RR 的 95% 置信区间线性放大	区间 5–17 分钟

所以界面上这一项旁边有一行小灰字：`推算值 · 长期少睡才成立`，点开是完整路径加三个前提——只长期成立、睡眠与死亡率呈 U 型、原研究为观察性未证因果。

> **原文给的数和折算出来的数，不是一回事**

> 不能让用户以为「每少睡一小时少活 11 分钟」是某份研究的结论。它是我的推算，而推算可以被质疑——这正是它必须写出来的原因。

4.3 铁律三：只在推演尺度内成立的数，才允许进推演

推演卡最长看一年。睡眠那个 11 分钟是「一辈子」摊出来的，放进一年的推演里，等于把一辈子的代价说成一年的代价——虽然数字一样，含义差两个数量级。所以 `forecast.js` 里有一份白名单：

```
const ELIGIBLE = ['smoke', 'alcohol', 'sit'];
```

睡眠**有数字也不进**。同样被挡在外面的还有含糖饮料、快走——方向可信，但没有可信的折算口径。

4.4 三个刻意的「不做」

三条否定式规则，比正向功能更重要

不做	具体表现	为什么
不抵消	抽烟 + 运动，心肺部位同时出现「不利」和「有益」两条结论，分别显示。	合成净值等于替用户做了一个他没有授权的换算：「跑了三公里」能不能抵「两支烟」？没人能回答。
不猜	没收录的行为显示「当前未收录对应结论」，不标为安全。	「未收录」不等于「无害」。把空白读成安全，是这个品类最常见的谎。
不插值	没有可信口径的行为，宁可只给方向不给分钟。	两个已知点之间画直线，看起来像科学，实际是我编的。

这三条在界面上的具体体现：正向行为（早睡、走路）**刻意不给分钟数**，只写「已记下」。因为一旦给出「− 30 分钟」，用户就会自然地去算「今天抽三支、走了半小时，扯平了」——那是产品最不该鼓励的算法。

05 / INTENT

意图控制：AI 协同怎么做的

这一节回答一个问题：**在这九天的开发里，AI 到底做了什么，我又做了什么。** 结论先说：AI 承担了绝大部分实现，但它提出的**方向性判断里，有相当一部分被否掉了**——而每一次否决都留下了原始记录。

5.1 角色分工：谁定方向，谁写实现

三种角色，边界清楚

角色	负责什么	不负责什么
我	提出意图、给出验收标准、否决方向、最后拍板	不写实现细节——但每一处改动我都知道它改了什么
AI（实现）	写代码、跑测试、量像素、写文档初稿	不决定产品该长什么样，不决定数字口径
AI（评审）	提出问题、给方案对比、指出我没想到的风险	不替我签字——它说「已完成」不算完成，要我看过

有一条经验值得单独说：**AI 最危险的地方不是写错代码，是写对了但方向错了。**代码错了测试会红，方向错了要等到用户看见才知道。所以我把大量精力放在「先让它讲清楚打算怎么做、我否掉再动手」上，而不是「做完再改」。

5.2 证据怎么留：三条链都能独立核验

比赛要求「≥3 个核心功能迭代的完整 Prompt 链」，且「整理稿不算原始证据」。所以每条链都留了三样东西：

一条 Prompt 链的四种证据形态

形态	是什么	能证明什么
在线原文	公开分享页链接，匿名可访问、无需登录	内容没有被改写；时间戳存在
图文归档	1440×4700 ~ 6900 的滚动拼接长图	页面原貌，含界面细节与顺序
全文抽取	`.txt`，正文原样导出	可全文检索、可逐字比对
可复现对照	同一段操作，在改动前后的两个提交上各跑一遍截图	改动确实发生过；`git archive` 出来自己就能重跑

会话本身带时间戳，四轮对话依次落在不同分钟上，**顺序是平台记录的，不是事后整理的。**这一点在 `docs/prompt-log/时间戳来源核验.md` 里有专门的上下对照截图：不注入任何样式时抽取的文本与归档逐字一致；仅靠鼠标悬停，时间戳就自然出现。

> **三条链的位置**
>
> `docs/prompt-log/sleep-ledger`（2026-10-22）· `dark-to-light`（2026-10-21）· `merge-pages`（2026-10-21）。索引在 `docs/prompt-log/00-索引.md`，每条都标了对应提交号，可以 `git show` 还原。

第四种形态用在 `today-ledger` 与 `evidence-gap` 两条链上。它们发生在 10-20，当时没做分享快照，拿不出原始留痕，所以材料里明确标为「可复现对照」，不冒充原始证据。

5·3 案例一：睡眠折算——我诊断错了病

> 用户 · 2026-10-22 20:23
>
> 点睡眠功能的时候上面数据显示缺，这个能改吗

> 用户 · 20:26
>
> 这个数据为什么显示的是缺 是因为数据缺失还是什么 能不能补上

我第一遍把它当成了「数据缺失」问题，准备去找一份新研究补一个数字进来。

用户第二句追问点破了实际诉求：「为什么显示缺」和「能不能补上」是两个问题。界面上那个「缺」字，读者读到的是「这个行为没什么事」，而不是「我们没有可靠口径」。

这是表达问题，不是数据问题。这一处纠正决定了后面所有取舍：优先修的是「怎么让人看懂这个数字是怎么来的」，其次才是数字本身。

> 用户 · 20:28 · 给出验收标准
>
> 我认为可以适当加一些提示词 再加上推测的时间，要不然用户可能把这个行为所造成的影响给弱化了

于是这一轮的产出不是「补了一个数」，而是**三条铁律**（第 4 章）加一个 `hint` 字段。数字从「缺」变成 11 分钟，但更重要的是旁边那行字。

> **复盘**
>
> 用户说「弱化」，我第一反应是「数字太小，需要放大」。实际是「没有说明，所以读者不知道该怎么信」。**数字不够大和数字没依据，是两种病，药不一样。**

5.4 案例二：深色翻浅色——我把成本当成了决策变量

> 用户 · 2026-10-21 16:08
> 我想把这个深色的界面改一下，我认为深色的会给用户带来严肃和压抑的感觉

我的回复开头是：**「我不反对你的感受，但我对原因的判断和你不太一样」**。然后给出分析——压抑来自**近黑 + 冷青 + 高对比**三者叠加，而不是深色本身；改浅色不是唯一解法。接着给了一张三版对比（A 现状 / C 暖深 / B 浅色）和一张时间账：选 C 两小时，选 B 要两天，而且截图和视频全部要重拍。

> 用户 · 16:28
> 抛开时间和别的因素，从公正客观的角度上说你认为整体色调是什么颜色才更合适

这一问把讨论从「省不省事」拉回到「产品该是什么样」。我这才给出五条论证，结论是浅底。

> 用户 · 16:36 · 否掉我的成本论证
> 这是参赛作品，不能为了省事就不了了之，那就按你这个浅色方案来吧

我一开始把「成本」当成了决策变量，而用户要的是「正确」，成本只是执行约束。这句话我记在了项目记忆里，因为它不只在这一次成立。

执行过程还有两个真实的坑：

- **改错文件，白干半小时。**先去改了 `artwork/anatomy-base.js`，截图页面毫无变化。查引用才发现那个文件没有任何地方引用，是早期废弃草稿，真正在跑的是根目录 `anatomy.js`。教训：改之前先确认文件在被谁引用，不能按文件名猜。
- **三版配色全被否，因为「心脏不能没有红」。**青绿同族——心脏变深青，看起来像病变；铜色单色——像铜版画挺好看，但心脏变褐，像标本；灰阶——肺、肝、肠糊成一团。最后定的是「外壳浅灰成白瓷，器官保留色相」：用明度反转公式整体翻，再按反转后的明度压饱和。

> **复盘**
> 用户提出的**感受**（压抑）比我的**归因**（近黑 + 冷青 + 高对比）更接近真问题。归因可以错，感受不会。产品决策里，用户的感受是数据，我对感受的解释只是假设。

5.5 案例三：两页合并——「割裂」被误诊为视觉问题

> 用户 · 2026-10-21 17:17
> ai 所占的部分会不会太少，还有「身体探索」感觉与「今天」有点割裂 咱们探讨一下

注意「咱们探讨一下」——这一轮要的不是动手，是判断。我没有直接给方案，先把两个问题分别诊断。

「AI 占的部分太少」——不是字数问题

AI 这层在工程上并不薄：本地规则引擎、在线客户端、服务端流式代理，还有白名单校验、失败降级、冷却重试。但它作用边界很清楚：**拿已经算好的数字，换一种说法。**所以评审只要问一句「把 AI 拿掉会怎样」，答案就是「同一句话朴素一点」。**AI 成了修辞，不是判断。**

「割裂」——不是长得不像，是两套数据

两页在合并前的实际差异

	今天页	身体探索页
数据	真实记录（账本）	假设情境，默认写死「抽烟 1 支 / 1 个月」
单位	时间（分钟）	器官（肺、肝、心）
交互	点一下就记	先设定情境，再看影响

长得不像只是表面。这个诊断直接决定了做法：**不是去统一样式，而是去统一数据。**我同时否掉了一个看似合理的方向——「两套人形（剪影和写实解剖）应该统一」。理由是：剪影放大到 500px 会显得简陋，写实解剖缩到 136px 会糊成一团。分层是对的，该统一的是数据和颜色，不是形状。

我给了三条桥：数据桥（探索页默认情境取今天页的真实记录）、AI 桥（让 AI 从候选行动里挑一件并说明理由）、文案桥（两页各补一句关系说明）。

> 用户 · 17:41 · 选了第四条路
>
> 能不能把今天和身体探索两个部分合并？不要今天的那个身体了 因为我看「今天」这个页面比较空，而「身体探索」又有过多的信息了

用户没有选三条桥中的任何一条，而是提出了更彻底的做法：**不搭桥，合并。**理由说得比我清楚——今天页太空，探索页信息过载，两个极端放在一起正好。

> 用户 · 17:57 · 纠正一个错误前提
>
> 先这么做，推演是不是由 ai 完成的，如果是就保留……还有就是那个人体模型的头部是不是太长了

这一问很关键：**如果推演是 AI 算的，那它就是不可复现的，应该砍掉。**实际不是——推演是权重表逐日累加，AI 只换措辞。所以推演留下了，理由写进了第 3.4 节。同时「头部太长」这句开启了一轮人体比例的返工。

> **复盘**
>
> 「割裂」的常见误诊是视觉不统一，真病因是两套数据。**先统一数据源，样式问题自己消失。**这次合并之后，全站只剩一处数据转换，之后所有「两边对不上」的问题就再没出现过。

5.6 被否掉的提议清单

这份清单比成功了什么更能说明意图控制。以下每一条都是 AI 提过、我否掉的：

方向性否决（部分）

AI 的提议	否掉的理由	替代方案
Next.js + Vercel 重写	会让 AC-24、AC-28、零支出三条同时作废，且 46 个文件重写是纯损耗	继续原生，AI 走服务端代理
用综合负荷分替代身体年龄	安全但失去唯一有力的总量表达	改分钟制为主要表达，负荷分退为辅助
先上「暖深色」过渡，不改全站	把成本当成决策变量，而用户要的是正确	直接全站翻浅色（2 天，截图视频重拍）
器官统一改成青绿色系	心脏变深青像病变；单色像铜版画；灰阶分不清器官	外壳浅灰、器官保留色相
两套人形（剪影 / 写实）统一成一套	剪影放大简陋，写实缩小糊成一团	保留分层，只统一数据与颜色
详情弹窗解释「为什么不折算成分钟」	用户要看的是这个行为本身是什么，不是折算规则的免责声明	文案改 A 版：解释行为，折算口径退到最后一行
给不给分钟数混在一句「数据不足」	正向行为是**刻意**不给（防「早睡抵一支烟」），含糖饮料是**确实没有**可信口径	分两类写清楚，不能混成一句

5.7 我怎么验收 AI 的输出

一条规矩：**不看它说「已完成」，看像素和测试。**过程中被这条规矩救回来过很多次，最典型的一次：

> 还是一样的，在动画的最后有点白有点空荡了，然后在动画的最后还是有些卡顿
>
> 用户，2026-10-26

当时的「已完成」是：关闭动画从 FLIP 改成容器收拢、内容反向缩放、0 慢帧。数字上全都达标。但用户看到的是「最后有点白、有点空」——因为文字在 60 毫秒就退场了，剩 300 毫秒只有白框在缩。

测试测的是「有没有按预期执行」，用户看的是「最后那 200 毫秒长什么样」。这两件事不重合，所以视觉改动必须逐帧看过才算完成。具体方法见 7.3 节。

另外两条硬规矩：

- **差值小于 1 像素不提交。**人眼对位移极敏感，一像素在边缘上就看得见东西挪了一下。
- **判断颜色一律采像素，不看眼睛。**截图取色比肉眼可靠，尤其在这种大面积低对比的浅色页面上。

06 / TIMELINE

演进时间线

2026-10-18 到 10-26，九天，86 次提交。 平均每天十次，最多的一天（10-21）三十二次——那天在翻浅色、合并页面、重画人体。 没有一次「initial commit 含全部功能」。

按天归纳的演进，每条都能 `git show` 还原

日期	提交	这一天做了什么，以及为什么
10-18	7	把 V0.1 导入作为版本化基线（不是从零开始，是从「已经能跑但有方向问题」开始）；随后立刻推倒重做 V2 的信息架构，因为单屏同时出现记录、负荷、身体年龄、恢复轴，用户无法判断该先看什么。
10-19	7	器官特写：点肺部后镜头沿单轴推进到器官内部。修掉一个隐蔽 bug——文字按钮被业务渲染替换后点击失效。
10-20	16	关键的一天。记一笔有了物理去向（粒子飞向器官并点亮）；接通解读层，本地规则打底、在线增强可选；新增零依赖服务端；补了两条数据的出处，但发现都不能照抄公式。
10-21	32	最大的一天，三件事并行：① 全站从深色翻成浅色（含人体外壳去暖肉色、面板底色改由 vfx 变量接管）；② 今天与身体探索合并成一页；③ 人体按 8 头身重画、头部压短、五官反复调。还有几个稳定性修复：缓存策略、失败冷却 60 秒、HTTPS 配置不被初始化脚本覆盖。
10-22	3	睡眠折算口径。建立三条数字铁律（第 4 章），数字从「缺」变成 11 分钟 + 推算路径。
10-23	5	发现归档图里时间戳是隐形的（父容器 `opacity:0`），重截并强制展开后重新归档，同时补了核验文档。
10-24	7	补回眼睛的眼区，让用眼类行为在人体上有落点；屏幕时间改映射到含眼表的资料库行为——让文字和画面说同一件事。粒子到达时不再给器官加视觉修饰，嘴的颜色全程不动。
10-25	4	按外部评审六条意见改今天页。最大的改动是「不再替用户举例」——空账本不显示数字，不预选行为。
10-26	5	资料库卡片可点开（详情弹窗，含容器收拢动画）；今日计划页标题改成对人说话；修掉打开弹窗时整页横跳 7.5px、以及右侧滚动条凭空消失。

为什么提交这么碎

比赛评分里「过程可复现与演进链路」占 25%，而评委会直接看 log 判断演进真实性。所以一次提交只做一件可独立描述的事：一个器官交互、一条规则、一次视觉调整、一组测试，各自一个 commit。提交信息写清「改了什么 + 为什么」，禁止「update」「优化一下」这类零信息量描述。

07 / QUALITY

质量保障

没有 CI，没有 QA 团队，所以可靠性靠三件事兜： **纯函数测试、无头浏览器探针、逐帧看动画。** 三者分工明确——前两个管「对不对」，第三个管「看起来对不对」。

7.1 85 项单元测试，不需要浏览器

因为计算层是纯函数（不碰 DOM、不碰网络、不碰 localStorage），测试可以直接在 Node 里跑，一条命令：

```
npm test        # 等价于 node --test
# tests 85
# pass 85
# fail 0
```

八个测试文件覆盖的范围

文件	锁住的是什么
`ledger.test.cjs`	折算、周额度、逐日合计、器官落点；外加两条断言要求 `W.validate()` 返回空数组。
`interpret.test.cjs`	本地解读规则的输出结构；账本为空时不编造句子。
`domain.test.cjs`	输入匹配与日期归档。含「不抽烟」不得匹配到抽烟、跨午夜时昨天记录不得计入今天。
`endpoint.test.cjs`	各家 OpenAI 兼容接口的路径拼接：只给域名要补 `/v1/chat/completions`、已经带 `/v1` 的不能重复拼、智谱是 `/api/paas/v4`、百炼是 `/compatible-mode/v1`；空值不抛异常、回落到默认路径。
`ai-offline.test.cjs`	断网时前端不报错、不空屏，标签退回「本地解读」。
`anatomy-coverage.test.cjs`	每个可交互器官都在 SVG 里有对应节点、有等效文字按钮（AC-12）。
`organ-focus.test.cjs`	器官聚焦的坐标换算与缩放边界。
`prompt.test.cjs`	提示词拼装。**锁的是「数据里没有的，不许向模型要」**：没有昨天的记录就不许要求做前后比较，没有「只记录」条目就不许点名它们，输入里也不许出现带负号的数字——否则模型照抄就会说出「比昨天多损耗 −200 分钟」这种没有依据的话。

一条自己定的规矩

修一个 bug 就要补一条对应用例，除非该行为无法在不引入 DOM 的前提下测试（此时在 `TESTING.md` 里记录原因）。所以测试数量只增不减——这一轮之前是 80 项，现在是 85 项。

7.2 无头浏览器探针：不看截图猜，问 DOM 要数

视觉和布局问题无法用单元测试覆盖，但也绝不能靠「截个图看着差不多」。所以写了一组 CDP（Chrome DevTools Protocol）探针脚本，直接连浏览器问数据：

四类探针，各自回答一个问题

脚本	回答什么	例子
`cdp-measure`	有没有元素越界	390px 窄屏下是否出现横向溢出（看 `clientWidth`，不看 `innerWidth`）
`cdp-boxes`	子元素的盒子在哪	右栏三个按钮的右边缘是否对齐
`cdp-eval`	任意表达式	用 `elementFromPoint` 做命中测试：这个位置到底能不能点到
`probe-*`	整条功能的端到端断言	详情弹窗 50 项：内容、计划联动、三种关闭方式、390 窄屏、控制台 0 报错

探针最大的价值不是「跑通」，而是**能推翻我自己**。有一次它报了溢出，我差点照着改布局；量完 `getBoundingClientRect()` 发现实际零溢出，是 `--dump-dom` 拿不到注入脚本改过的 DOM 造成的假象。**如果当时照着假象改代码，会把一个好好的布局改坏。**

7.3 逐帧验证：动画不能只看最后一张截图

电影里 200 毫秒等于五帧。用户说「最后有点卡」「最后有点白」，指的是那五帧里的某一帧，而 `Page.captureScreenshot` 本身有 100~200 毫秒延迟，**根本抓不到这个过程**。

解决办法有两个：

- **冻结动画。** `a.pause(); a.currentTime = N`，一帧一帧看。只冻需要看的那一层，不要连遮罩一起冻。
- **用 `Page.startScreencast` 抓真实渲染帧。**浏览器自己推帧，带时间戳。滚动条那次抓了 157 帧，把打开和关闭全程看了一遍。

具体怎么判？不是「看着还行」，是**取像素算差值**。滚动条那一列的亮度与相邻页面底色的差值，全程稳定在 −25 到 −35 之间，没有一帧跳到 0——这才叫「滑块自始至终都在」。

157 帧里，滑块所在那一列和它旁边 12px 的页面底色，亮度差始终在 −25 到 −35 之间。关闭动画结束的那一帧也不例外——**它没有突然跳成 0**，也就是滑块自始至终都在。这一条不可能靠单张截图证明，只能靠整段帧序列。

7.4 像素级验收：两个来源互相印证

最难验的一次是「弹窗打开时滚动条不许消失」。做法是：锁背景期间在原位画一条替身，替身的几何要跟系统那条一模一样。

验收方式不能是「看起来一样」，得两个来源对撞：**系统那条画在哪，从截图数像素；替身画在哪，问 DOM 要矩形。**

五组用例（本地与线上各跑一遍）

用例	结果	说明
1440 宽 · 页面在顶部	起止行列一致	滑块长度与位置都对齐，切换那一帧看不出差异。
1440 宽 · 滚到 600	起止行列一致	滚动位置变化时，长度不变、位置按比例移动，两边同步。
1280 宽	起止行列一致	页面变窄后内容高度变化，替身的长度跟着变。
1080 宽	起止行列一致	接近单栏断点，布局重排后的复核。
390 宽	修正后一致	**这一组暴露了 1px 误差**：位置用四舍五入会多算一格，改成向下取整才和系统那条重合。

顺带记两个工程细节，都是踩出来的：

- `Page.captureScreenshot` **带 clip 时坐标是页面坐标，不是视口坐标。**滚到 600 再截，会截到别处去。整屏截再自己裁。
- `--window-size=W` **的真实视口是 W−15。**判断溢出要看 `clientWidth`，用 `innerWidth` 会永远觉得没溢出。

部署与复现

两条路径都要通：一条是评委在自己机器上十分钟跑起来，一条是我把改动安全地推到线上。 前者是评分要求，后者是开发日常——两条都必须可靠，否则演示当天就会出事。

8.1 本地：十分钟，不需要 `npm install`

README 里的分步流程（2026-10-25 在空目录实跑核对过）

用时	操作	应该看到
0 分钟	`node -v`	v18 及以上
1 分钟	`git clone && cd`	能看到 `index.html`、`app.js`
2 分钟	`npm start`	终端输出 `BodyLens: http://127.0.0.1:4173`
3 分钟	浏览器打开地址	首屏「还没开始」，右栏一具人体
5 分钟	点行为 → 点数量	大数字变成 `20 分钟`，下面「大约等于 一集短剧」
7 分钟	继续往下滚	解读 / 推演 / 近七天三块都在，空账本时都不给数字
9 分钟	断网，再记一笔	功能不受影响，标签仍是「本地解读」
10 分钟	点右栏任一器官	器官详情：功能、关联行为、证据与来源

`package.json` 里只有两个脚本，都指向仓库自带的文件，不引用任何第三方包。没有 `node_modules`，也**不需要**执行 `npm install`。

> **这张表是锚点，不是宣传**
>
> 表里写的必须和产品实际一致。对不上时**修产品，不要改表迁就产品**——因为表是评委第一次接触这个项目时看到的东西，它错一次，后面所有验证都会被怀疑。

8.2 线上：一台最小配置的云主机

主机	腾讯云轻量应用服务器，Ubuntu 22.04，2 核 2G
目录	`/opt/bodyledger`，属主 `www-data`
Web	Nginx 反代 `127.0.0.1:8080`，静态文件直出
进程	systemd `bodyledger.service` 跑 `server/proxy.cjs`（解读代理）
证书	Let's Encrypt，自动续期
域名	body-ledger.com

三个配置细节值得说，因为它们各自对应一次真实故障：

- `proxy_buffering off`。解读接口是流式返回（SSE），Nginx 默认会缓冲，表现是「一直转圈，最后一次性全出来」。关掉之后才有逐字出现的效果。
- **JS/CSS 用 `expires off` + `Cache-Control: no-cache`，靠 ETag 走 304。**既保证我发布后立刻生效，又不会每次刷新都重新下载。
- **发布后必须 `chown www-data:www-data`。**用 root 复制文件会让属主变成 root，Nginx 读不到，直接 403。这个坑踩过一次，页面白了一次。

8.3 发布流程：每次都验校验和

```
# 1. 本地打包（只发三个文件，不碰 server/ —— 线上有 .env 和 prompt.cjs）
tar czf /tmp/release.tgz app.js index.html styles.css

# 2. 上传并解压到临时目录
scp /tmp/release.tgz ubuntu@<host>:/tmp/
ssh ubuntu@<host> 'tar xzf /tmp/release.tgz -C /tmp/rel'

# 3. 复制、改属主、两端对比 md5
sudo cp /tmp/rel/* /opt/bodyledger/
sudo chown www-data:www-data /opt/bodyledger/app.js ...
md5sum /opt/bodyledger/app.js      # 与本地一致才算发布成功

# 4. 确认服务活着
systemctl is-active bodyledger
```

为什么要 md5 对比：**「文件传上去了」和「文件是对的」是两件事。**慢网络下 scp 断掉不会报错，只会传一半。发布后立刻用探针打一遍线上，是最后一道关。

09 / LIMITS

已知局限

这一节列我自己知道有问题、并且**还没有解决**的地方。 写它们的理由很简单：这份文档如果只讲做成了什么，它就不值得相信。

9.1 「时间损耗」这个隐喻，可能经不起追问

产品把行为的后果折算成时间。这个隐喻好用，但它有一个隐含假设：**时间是一个可以被扣减的账户，而且扣掉的量可以逐次累加。**

真实的流行病学不是这样运作的。寿命损失是人群统计意义上的期望值缩减，不是一个人身上有一本余额。当一个人这个月抽得少、下个月抽得多，模型不会按我的加法走。

目前的应对是：所有数字都标着「群体平均值」「用于累积对比，不用于推测你还能活多久」。这是缓解，不是解决。**如果要做下一版，我会把「账」从一个连续累加的池子，改成「最近 7 天的加权窗口」——但那需要先想清楚单位怎么表达，现在没有答案。**

9.2 账本是「今天的」，健康是「长期的」

产品的核心界面是今天。今天记了十支烟，数字就是 200 分钟。但健康影响不是今天的——它是十年累积的。这个时间尺度的错位，是「用时间结账」这个机制自带的：越容易理解，越容易失真。

推演卡是补丁：把今天的量摊到一年。但它只能是补丁——推演里只有三条行为有资格进来（第 4.3 节），大部分行为在推演里是空白的。用户看到空白，会以为「这个行为不重要」。

9.3 三十条行为里，只有三条能进推演

这是三条铁律的直接后果，也是最明显的功能缺口。用户想算「如果我每天喝两杯奶茶，一年等于多少天」，产品给不出——因为含糖饮料没有可信的折算口径。

我可以编一个数：一克糖多少秒。但那会让整个推演失去可信度。**宁可留一个洞，也不填一个假数。**洞的代价是：产品在「全面」这一项上明显不合格。

9.4 我在自己给自己做裁判

整个过程里，提出标准的人、实现的人、验收的人，都是同一个（或者说，我和 AI 这两个）。缺一个真正的第三方。产品里那些「已验证」的标记，验证的是**「符合我自己定的标准」**，不是「符合医学标准」。

资料由我在开发阶段核对公开原文，**尚未经临床专家或解剖专业审校。**这一点在产品设置页里明写着，没有藏。

9.5 移动端没做完

390px 宽能正常使用：单栏重排、无关键文字裁切、弹窗可用（50 项探针里专门有一组窄屏）。但它不是「为手机设计的」，是「在手机上不坏」。触屏手势、单手可达区域、滚动惯性这些都没专门处理。

9.6 最重要的那个数字，出处反而不完整

香烟那一项是全产品的基准——首屏默认展示它，一句「20 分钟」撑起整个产品的可信度。但这个数字的来源是**群体平均寿命损失摊算**，和睡眠那项一样是折算值，而不是某份研究里直接写着「一支烟等于二十分钟」。

它的 caveat 写着：「群体平均值（男 17、女 22）。个体差异很大，用于累积对比，不用于推测你还能活多久。」这是产品里最短、也最重要的一句限制说明。

> **还有一件没想清楚的事**
>
> 器官的可交互集合是动态的：只有「在资料库里出现过」的器官才显示。胃目前零条关联，所以不显示；一旦补上证据，它会自动回来。这个机制是对的（不删数据、只改规则），但它带来一个副作用——**用户看到的身体，取决于我收录了多少证据**。一个缺了胃的人体，可能会被读成「胃不重要」。目前没有好的表达方式。

APPENDIX A

附录：PRD 验收项与当前实现的偏差

PRD 第 10 节列了 28 条验收项。产品经过几轮迭代之后，其中 5 条**已经不适用了**—— 不是没做到，是产品变了。这一节把偏差逐条说清，而不是悄悄改 PRD 去迁就实现。

五条已失效的验收项，以及为什么

编号	原文要求	当前状态与原因
AC-02	输入 17、空值、小数等年龄并尝试分析，给出字段说明	**不再收集年龄。**产品改成面向「18 岁及以上一般成年人」的科普工具，不做个人化风险计算，因此不需要年龄这个输入。删掉它同时消掉了一整类「你输入错了」的错误提示。
AC-06	单项模式选两项、综合模式添加行为，各自情境互不覆盖	**单项 / 综合两种模式已合并。**现在只有一条路径：账本记录什么，情境就是什么。两套模式被砍掉的原因和「两页合并」相同——两套数据必然不同步。
AC-07	切换每天 1 支 / 2 支及持续时间	**不再手动设情境。**情境由账本的真实数量推导（`today.js` 的 `scenario()`）。手动设「每天几支、持续多久」和「今天记了几支」并存，会出现两个互相矛盾的数。
AC-08	修改情境但不点更新时，旧结果标明原情境并提示待更新	**已无此状态。**没有「待更新」这个中间态了——账本一变，结果立刻重算。这是删掉手动情境设置之后的自然结果。
AC-11	切换消化 / 呼吸 / 泌尿等系统和正背视图	**系统筛选与正背视图已删除。**理由是它们在替用户决定看什么：一次只显示一个系统，会让用户以为其他器官今天没事。现在全部器官同时在场，只标出今天受影响的那几个。

为什么不改 PRD

PRD 是需求基线，改它等于把自己的验收标准往实现上靠。这几条偏差在 `PRD.md` 的变更记录里逐条登记，理由写在这里——**哪一条被砍、什么时候砍的、为什么砍，都能查。**剩下的 23 条按 PRD 第 10 节原文执行，逐项的实际验证记录在 `TESTING.md`，本文档不重复声称。

A.1 三条硬指标的实际验证

AC-24 / AC-28 / AC-27 的验证方式

编号	怎么验的	结果
AC-24	起本地服务器 → 断网 → 记一笔 → 点器官 → 打开计划页 → 导出备份	全部可用。页面无任何外链字体、图片、脚本，唯一网络请求是可选的 `/api/interpret`，断网时返回失败并自动退回本地文案。
AC-28	空目录 `git clone` 后按 README 表格逐条走一遍，计时	约 6 分钟到首屏可用（不含读文档时间），无 `npm install`、无密钥。
AC-27	系统开启「减少动态效果」，再打开页面里的动效开关，重复五个核心操作	悬停、点击、记账、详情、推演全部可用；无持续闪烁。降级路径单独写了高特异性规则，因为全局 `*{transition:none}` 会吃掉普通规则。

A.2 一句话后记

这个项目最花时间的部分，不是把功能做出来，是**把不该做的东西拿掉**。身体年龄、健康总分、损伤百分比、单项综合两种模式、系统筛选、正背视图——每一个拿掉的时候都会让产品「看起来少了一点」，但每一个都换回了同一样东西：用户不会因为它而相信一件不成立的事。

技术选择也是同一件事的另一面。不用框架不是保守，是因为这个产品的核心资产是**可被验证**：数字能被复算，说明能被追溯，代码能在十分钟内跑起来。这些都不需要框架，也都经不起「先装 400 个包」。